2강. JavaScript 및 Node.js

JavaScript 작성하기

- **자바스크립트 언어**
  - 정의: 웹 브라우저에서 실행되는 대표적인 스크립트 언어.
  - 역할: 웹 페이지에 **동적 기능(Interactive)**을 추가하는 데 사용.
    (HTML → 구조, CSS → 디자인, JavaScript → 동작)
  - **역사**

    1995년: Netscape에서 Brendan Eich가 10일 만에 JavaScript 개발

    1997년: ECMAScript로 표준화

    2008년: 구글 V8 엔진, Chrome → 성능 혁명

    2009년: **Node.js** → 서버에서도 활용

    2015년 이후: **ES6(ES2015)** 등장, 현대적 문법 확립, 프레임워크 시대 개막

JavaScript 작성하기

- **특징**

인터프리터 언어
→ 별도의 컴파일 과정 없이, 작성하면 바로 실행됩니다.

브라우저 내장
→ Chrome, Edge, Firefox, Safari 등 모든 브라우저에서 지원합니다.

객체 기반 언어
→ 객체(Object)를 활용하여 데이터와 기능을 효율적으로 관리할 수 있습니다.

동적 타이핑
→ 변수 선언 시 타입 지정이 필요 없고, 실행 시점에 타입이 결정됩니다.

이벤트 기반 프로그래밍
→ 버튼 클릭, 입력, 마우스 이동 같은 이벤트에 따라 코드 실행.

JavaScript 작성하기

- **활용 분야**

웹 프론트엔드
버튼 클릭, 폼 입력 검증, 애니메이션, AJAX(비동기 데이터 처리)

웹 백엔드
Node.js 환경에서 서버 개발 가능

모바일 앱
React Native, Ionic 같은 프레임워크로 앱 개발

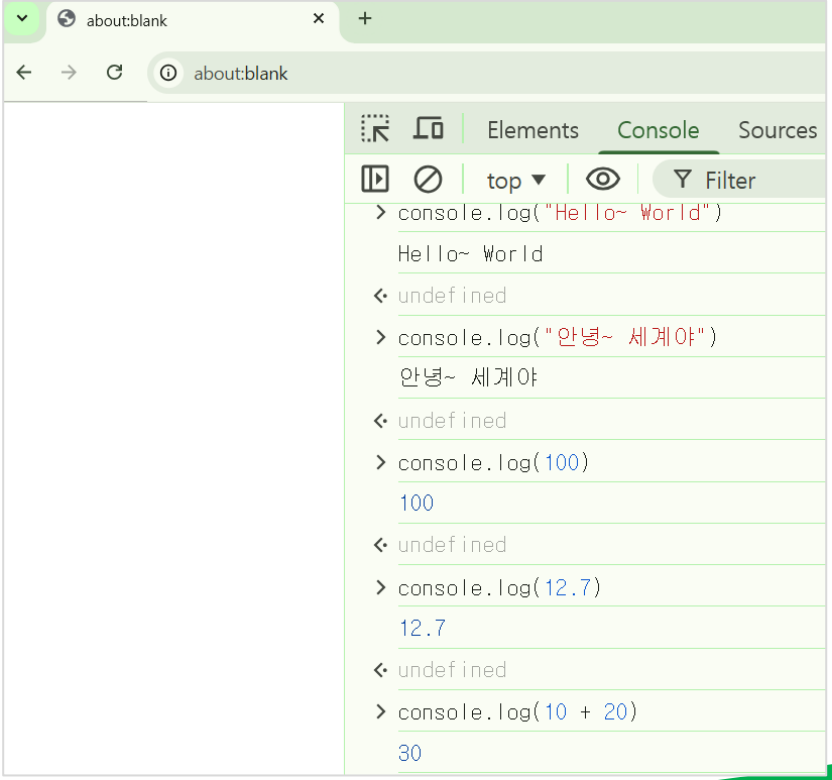
데스크탑 앱
Electron을 이용해 크로스 플랫폼 앱 개발 (VS Code도 Electron 기반)

게임 개발
HTML5 Canvas + JavaScript로 2D/3D 게임 제작

데이터 출력

콘솔창을 이용한 출력

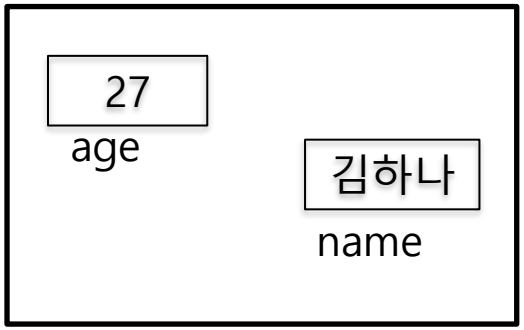
- 크롬 브라우저 > about:blank(빈 페이지 생성)
- F12(개발자도구) > Console
- **console.log() 메서드**



변수(Variable)

- **변수란?**
  - 프로그램에서 사용되는 자료를 저장하기 위한 메모리 공간(영역)
  - 할당받은 메모리의 주소 대신 부르는 이름 (메모리 주소: ac13bf00)
  - 프로그램 실행 중에 값 변경 가능, variable 이라 함

- **변수의 선언 및 초기화**
  - 변수 선언은 어떤 타입의 데이터를 저장할 것인지 그리고 변수이름은 무엇인지를 결정한다. **var, let** 키워드를 사용한다.
  - **(자료형) 변수이름**; var age; let name;
  - **(자료형) 변수이름 = 초기값;**
    var age = 27, let name = "김하나"



변수(Variable)

- **변수의 선언과 초기화**

let 변수명 = 값

- **변수명 작성시 주의**

- 변수 이름은 숫자로 시작할 수 없고, 공백 불허

틀린 예) var 3box; let sky 1234

- 변수 이름은 영문, 한글, 특수문자, 숫자의 결합으로 만든다.
- 영어 대소문자를 구별하며 예약어는 쓸 수 없다.

예약어) if, for, while 등

변수(Variable)

- 변수와 상수

```
<script>
  let name = "이정후";
  let age = 26;
  let isStudent = false;

  console.log("Name:", name);
  console.log("Age:", age);
  console.log("Is Student:", isStudent);

  // 값 변경(재할당)
  age = 27;
  console.log("Updated Age:", age);

  // Constant variable(상수)
  const birthYear = 1988;

  //birthYear = 1994; // Error: 재할당할 수 없음.
  console.log("Birth Year:", birthYear); //1998
</script>
```

variable.html

상수(Constant)

- **상수란?**

  한번 설정해 두면 그 프로그램이 종료 될 때까지 변경될 수 없는 값

- **상수의 선언과 초기화**

  const 변수명 = 값

- **상수 작성시 주의**

  - 값을 변경(재할당) 할 수 없다. – 오류 발생
  - 관례상 대문자 사용

호이스팅(hoisting)

- **var 변수와 호이스팅**
  - ES6 이전에는 변수를 선언할 때 var 예약어로 사용, 지금도 사용함
  - **호이스팅**: '끌어올린다'는 의미로 자바스크립트 해석기가 변수를 기억해 두어 선언한 효과를 가짐.
    즉, 선언하지 않고도 사용할 수 있고, 오류도 발생하지 않음

호이스팅(hoisting)

- **var 변수와 호이스팅**

```
<script>
  // var는 호이스팅되어 선언이 끌어올려짐
  console.log(x); // undefined
  var x = 5;
  console.log(x); // 5

  // let과 const는 호이스팅되지 않음
  console.log(y);
  let y = 10;
  console.log(y);

  console.log(z);
  const z = 15;
  console.log(z);
</script>
```

호이스팅(hoisting)

- **변수 비교**

구 분	선언하지 않고 사용	재선언	재할당
let	오류 발생	X	O
const	오류 발생	X	X
var	오류 없음(호이스팅)	O	O

자료형(Type)

- **자료형(type)**

  프로그램에서 처리할 데이터의 형태

  사용할 데이터의 종류에 따라 메모리 공간을 적절하게 설정해 주는 것.

구분	자료형	예시
숫자(정수, 실수)	number	let age = 16, let unit = 2.54
문자(문자열)	string	let season = "winter"
불리언(true/false)	boolean	let isMerried = true
배열	object	let seasons =["봄", "여름","가을", "겨울"]
객체	object	let date = new Date()

자료형(type)

- **숫자형**
  - 정수와 실수를 함께 묶어서 '숫자형 ' 이라고 한다.
  - typeof(숫자) 확인하면 - number
- **문자형**
  - 작은 따옴표나 큰 따옴표로 묶은 자료형이다.
  - typeof(문자) 확인하면 - string
- **불리언형**
  - true / false 값을 가지는 자료형이다.
  - typeof(문자) 확인하면 - boolean

자료형(type)

- **undefined**

자료형이 정의되지 않았을 때의 데이터 상태 : 변수를 선언할 때 아직 값이 할당되지 않은 경우

- **null**

데이터 값이 유효하지 않은 경우 : 변수에 값이 할당되었는데 잘못된 (유효하지 않은) 값인 경우

```
> let x
<· undefined
> age = prompt("나이 입력: ")
<· null
```

→ (취소한 경우 null이 들어감)

자료형(type)

- **배열**

  한 개의 변수에 여러 개의 값을 저장하는 자료형이다.

  대괄호를 사용하고 인덱스(위치)로 접근함

- **객체**

  한 개의 변수에 **다양한 자료가** 포함되어 있는 자료형이다.

  키와 값의 쌍으로 구성됨

  중괄호를 사용함

- **템플릿 리터럴**

  문자열과 변수, 식을 섞어서 하나의 문자열을 만드는 표현 형식이다.

  백틱(`)을 사용해 문자열을 만든다.

자료형(type)

- **자료형 예제**

```
  <title>자료형 예제</title>
</head>
<body>
  <script src="js/type.js"></script>
</body>
</html>
```

자료형(type)

- **자료형 예제**

```
//1. 숫자형
let age = 25; // 정수
let temperature = 24.5 ; // 실수

console.log("나이:", age);
// console.log("나이:" + age + "세");
console.log(typeof(age)); // number
console.log("온도:", temperature);
console.log(typeof(temperature)); // number

//2. 문자열형
let player = "이정후";
let city = '샌프란시스코';

console.log("선수 이름:", player);
console.log(typeof(player)); // string
console.log("도시 이름:", city);
console.log(typeof(city)); // string
```

type.js

자료형(type)

- **자료형 예제**

```
//3. 불린형
let isRaining = true;
let isSunny = false;

console.log("비가 오는가?", isRaining);
console.log(typeof(isRaining)); // boolean
console.log("맑은가?", isSunny);
console.log(typeof(isSunny)); // boolean

//4. undefined
let score; // 변수 선언만 하고 초기화하지 않음
console.log("점수:", score);
console.log(typeof(score)); // undefined
```

자료형(type)

- **자료형 예제**

```
//5. 배열형
let fruits = ["사과", "바나나", "오렌지"];
console.log("과일 목록:", fruits);
console.log(typeof(fruits)); // object (배열도 객체의 한 종류)

//6. 객체형
let person = {
  name: "우영우",
  age: 30,
  city: "서울"
};
console.log("사람 정보:", person);
console.log(typeof(person)); // object
console.log("이름:", person.name);
console.log("나이:", person.age);
console.log("도시:", person.city);
```

항과 연산자

- **항(operand)**
  - 연산에 사용되는 값

- **연산자(operator)**
  - 연산에 사용되는 기호

    예) 3 + 7 (3과 7은 항, '+'는 연산자)

- **연산자의 종류**

구분	연산자	연산 예
대입연산자	'='	num = 10
산술연산자	+, -, *, /, //, %	5 % 4
비교연산자	>, >=, <, <=, ==, !=	7 == 3
논리연산자	and, or, not	7 >=3 and 3 !=3
복합대입연산자	+=, -=, *=, /=	n += 1

산술 및 증감 연산자

◆ 산술 및 증감 연산자

연산자	연산 작업	설 명
+	result = n1 + n2	두 수를 더하기
-	result = n1 - n2	두 수를 빼기
*	result = n1 * n2	두 수를 곱하기
/	result = n1 / n2	두 수를 나누기
%	result = n1 % n2	두 수의 나머지
++	n++ (n = n + 1)	1 증가하기
--	n1-- (n = n – 1)	1 감소하기

연산자

◆ 복합 대입 연산자

연산자	연산 작업	연산 작업
+=	val += 10	val = val + 10
-=	val -= 10	val = val - 10
*=	val *= 10	val = val * 10
/=	val /= 10	val = val / 10
%=	val %= 10	val = val % 10

산술 및 증감 연산자

◆ 산술 및 증감 , 복합 대입 연산자

```
// 산술 연산자
let x = 10;
let y = 3;
console.log("x + y = " + (x + y)); // 13
console.log("x - y = " + (x - y)); // 7
console.log("x * y = " + (x * y)); // 30
console.log("x / y = " + (x / y)); // 3.3333...
console.log("x % y = " + (x % y)); // 1
console.log("x ** y = " + (x ** y)); // 1000

//증감 연산자
let b = 5;
console.log("b = " + b);  // 5
console.log("++b = " + (++b));  // 6 (전위 증가)
console.log("b++ = " + (b++));  // 6 (후위 증가)
console.log("b = " + b);  // 7

let c = 5;
console.log("c = " + c);  // 5
console.log("--c = " + (--c));  // 4 (전위 감소)
console.log("c-- = " + (c--));  // 4 (후위 감소)
console.log("c = " + c);  // 3
```

산술 및 증감 연산자

◆ 산술 및 증감 , 복합 대입 연산자

```
// 복합 대입 연산자
let val = 10;
val += 5;  // val = val + 5
console.log("val += 5 => " + val); // 15
val -= 3;  // val = val - 3
console.log("val -= 3 => " + val); // 12
val *= 2;  // val = val * 2
console.log("val *= 2 => " + val); // 24
val /= 4;  // val = val / 4
console.log("val /= 4 => " + val); // 6
val %= 4;  // val = val % 4
console.log("val %= 4 => " + val); // 2
val **= 3;  // val = val ** 3
console.log("val **= 3 => " + val); // 8
```

비교 연산자

◆ 비교 연산자

연산자	연산 작업	결과
>	10 > 5	true
>=	10 >= 5	true
<	10 < 5	false
<=	10 <= 5	false
==	10 == 10	true
!=	10 != 10	false
==	10 == '10'	true
!=	10 != '10'	false
===	10 === '10'	false
!==	10 !== '10'	true

'=='는 문자와 계산시 오류 발생

'===' 와 '!==' 사용

논리 / 조건 연산자

◆ 논리 연산자

연산자	기능
&&	두 항이 모두 참인 경우에만 결과값이 참.
||	두 항 중 하나의 항만 참이면 결과값이 참.
!	단항이 참이면 결과값은 거짓, 거짓이면 참

◆ 조건 연산자

(조건식) **?** 조건식이 true이면 실행 **:** 조건식이 false이면 실행

비교 / 논리 연산자

◆ 비교 및 논리 연산자

operator2.js

```
// 비교 연산자
console.log(5 == 3); // false
console.log(5 != 3); // true
console.log(5 == '5'); // true
console.log(5 === '5'); // false

// 논리 연산자
let a = 10;
console.log(a > 5 && a < 15); // true
console.log(a > 5 || a < 5); // true
console.log(!(a > 5)); // false
```

비교 / 논리 연산자

◆ 비교 및 논리 연산자

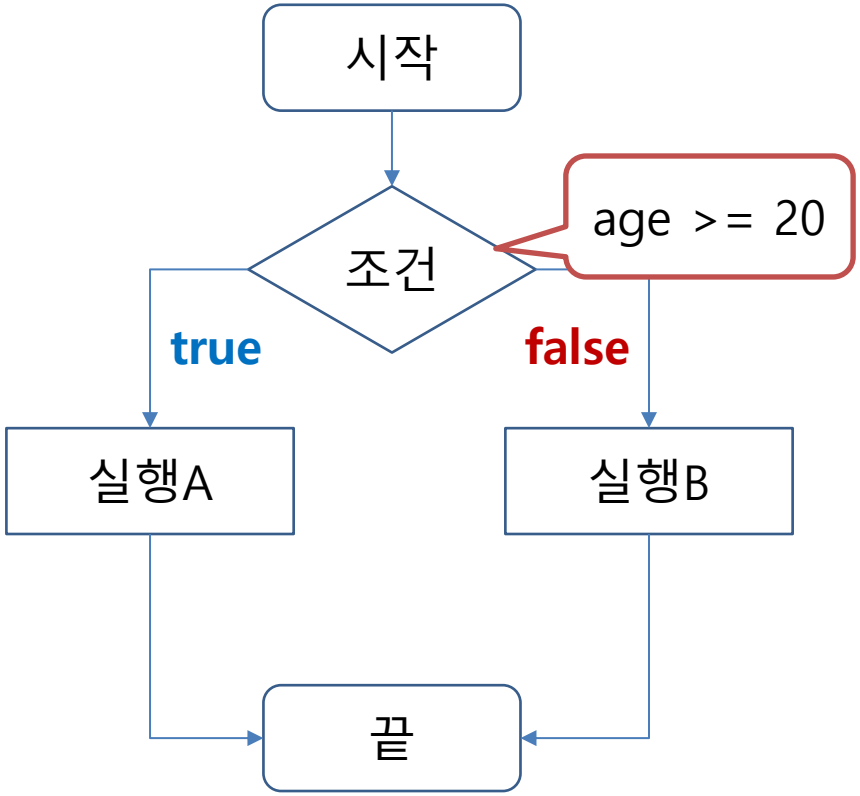
operator2.js

```
// 조건 연산자 (삼항 연산자)
let age = 20;
let isAdult = (age >= 20) ? "성인" : "미성년자";
console.log(`나이: ${age}, 결과: ${isAdult}`);

let num = 7;
let result = (num % 2 === 0) ? "짝수" : "홀수";
console.log(`숫자: ${num}, 결과: ${result}`);
```

조건문

❖ **조건문**

- 특정한 조건에 의해서 프로그램 진행이 분기되는 문장
- if문, switch문이 대표적임.



if 문

❖ **if 문**

true

```
if(조건식){
    실행문   //조건식이 true이면 실행
}
```

❖ **if ~ else문**

```
if(조건식){
  실행문1  //조건식이 true이면 실행
}else{
  실행문2  //조건식이 false이면 실행
}
```

if ~ else 문

❖ 짝수/홀수를 판별하는 프로그램

짝수/홀수 판별

13은(는) 홀수입니다.

```
<h2>짝수/홀수 판별</h2>
<div id="result"></div>

<script>
  //let num = 11; // 판별할 숫자
  // 사용자로부터 숫자 입력 받기
  let num = parseInt(prompt("숫자를 입력하세요:"));
  console.log(typeof(num)); // number
  const result = document.getElementById("result");
```

If ~ else 문

❖ 짝수/홀수를 판별하는 프로그램

```
  // 짝수/홀수 판별
  if(isNaN(num)){
    result.textContent = "입력이 취소되었거나 유효하지 않은 숫자입니다.";
  } else{
    if (num % 2 === 0) {
      //result.textContent = `${num}은(는) 짝수입니다.`;
      result.innerHTML = `<b>${num}</b>은(는) 짝수입니다.`;
      result.style.color = "blue"; // 짝수는 파란색으로 표시
    } else {
      result.innerHTML = `<b>${num}</b>은(는) 홀수입니다.`;
      result.style.color = "red"; // 홀수는 빨간색으로 표시
    }
  }
</script>
```

if ~ else 문

❖ 버튼 이벤트

짝수/홀수 판별

12은(는) 짝수입니다.

판별하기

```
<h2>짝수/홀수 판별</h2>
<div id="result"></div>
<button onclick="calculate()">판별하기</button>

<script>
  const calculate = () => {
    // 사용자로부터 숫자 입력 받기
    let num = parseInt(prompt("숫자를 입력하세요:"));
    const result = document.getElementById("result");
```

if ~ else 문

❖ 버튼 이벤트

```
    // 짝수/홀수 판별
    if(isNaN(num)){ //num ==null || isNaN(num)
      result.textContent = "입력이 취소되었거나 유효하지 않은 숫자입니다.";
    } else{
      if (num % 2 === 0) {
        //result.textContent = `${num}은(는) 짝수입니다.`;
        result.innerHTML = `<b>${num}</b>은(는) 짝수입니다.`;
        result.style.color = "blue"; // 짝수는 파란색으로 표시
      } else {
        result.innerHTML = `<b>${num}</b>은(는) 홀수입니다.`;
        result.style.color = "red"; // 홀수는 빨간색으로 표시
      }
    }
  }
</script>
```

다중 조건문

❖ **if ~ else if ~ else문**

```
if(조건식1){
    실행문1;  //조건식1이 true이면 실행
}else if(조건식2){
    실행문2;  //조건식2가 true이면 실행
}else{
    실행문3;  //조건식1,2가 모두 false이면  실행
}
```

if ~elseif ~ else

- 미니 프로그램 – 신호등

```
<h2>신호등 색상 판별</h2>
<div id="result"></div>
<button onclick="checkColor()">판별하기</button>
<script>
  const checkColor = () => {
    // 사용자로부터 신호등 색상 입력 받기
    let color = prompt("신호등 색상을 입력하세요 (빨강, 노랑, 초록):");
    const result = document.getElementById("result");
```

if ~elseif ~ else

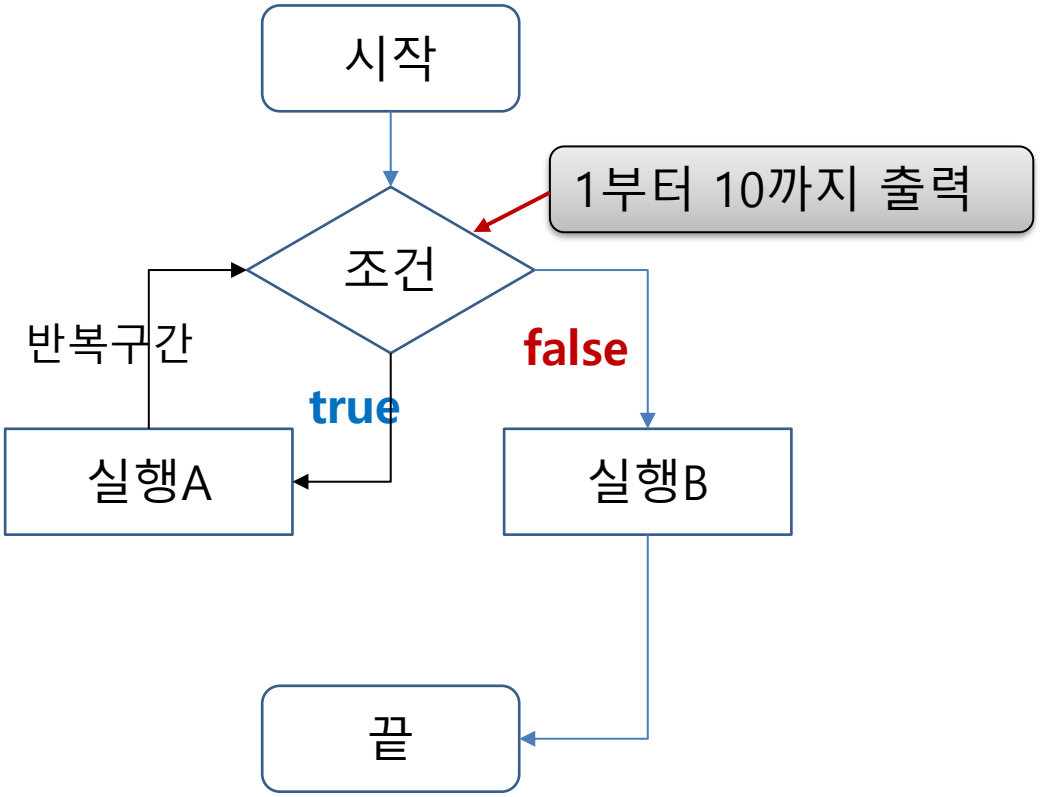
- 미니 프로그램 – 신호등

```
    if(color == null){
      result.textContent = "입력이 취소되었습니다.";
    } else {
      color = color.trim(); // 입력값 양쪽 공백 제거
      if (color === "빨강") {
        result.textContent = "멈춤";
        result.style.color = "red"; // 빨간색으로 표시
      } else if (color === "노랑") {
        result.textContent = "준비";
        result.style.color = "orange"; // 주황색으로 표시
      } else if (color === "초록") {
        result.textContent = "출발";
        result.style.color = "green"; // 초록색으로 표시
      } else {
        result.innerHTML = "유효하지 않은 색상입니다.<br> \
          빨강, 노랑, 초록 중 하나를 입력하세요.";
      }
    }
  }
</script>
```

반복문

반복문

- 주어진 조건이 만족할 때까지 실행문을 반복적으로 수행



반복문

- **while문**

```
let i = 초기값;
while(조건식){
    실행문;
    증감값;
}
```

while문

- **while문**

```
<!-- while문 -->
<script>
  let i = 1; // 초기값
  while(i <= 5){ // 조건식
    console.log(i); // 반복 실행할 코드
    i++; // 증감식
  }

  // 1부터 10까지의 숫자 중에서 짝수만 출력하기
  let j = 1;
  while(j <= 10){
    j++;
    if(j % 2 === 0)
      console.log(j);
  }
```

while문

- **while문**

```
// 1부터 10까지의 숫자 더하기
let k = 1;
let sum = 0;
while(k <= 10){
    sum += k; // sum = sum + k;
    console.log(`k=${k}, sum=${sum}`);
    k++;
}
console.log("1부터 10까지의 합:", sum);
</script>
```

```
k=1, sum=1
k=2, sum=3
k=3, sum=6
k=4, sum=10
k=5, sum=15
k=6, sum=21
k=7, sum=28
k=8, sum=36
k=9, sum=45
k=10, sum=55
1부터 10까지의 합: 55
```

while ~ break 문

- **while문 - if break문**

  반복문에서 break 문을 만나면 더 이상 반복을 수행하지 않고 반복문을 빠져 나옴

```
let n = 초기값;
while(true){
    if문
        break;
    실행문;
    증감값;
}
```

```
<body>
  <!-- break문 -->
  <script src="js/break.js"></script>
</body>
```

while ~ break 문

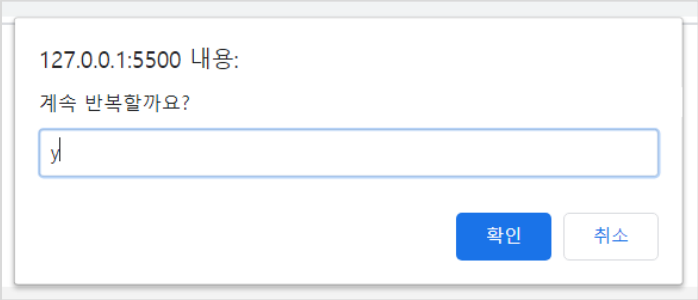
- while문 - if break문

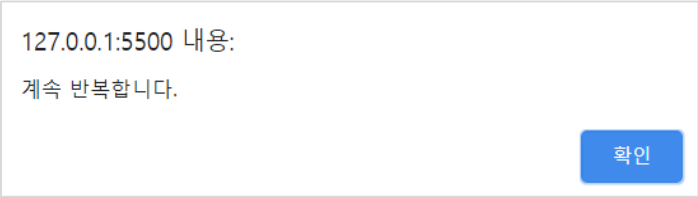
```
// 1부터 5까지 출력하기
let i = 1;
while(true){
  console.log(i);
  i++;
  if(i > 5){
    break;
  }
}
```

```
// 1부터 10까지의 짝수 출력하기
let j = 1;
while(true){
  if(j % 2 === 0)
    console.log(j);
  j++;
  if(j > 10){
    break;
  }
}
```

while ~ break 문

while문 - break문





```
while(true){
  let key = prompt("계속할까요?", 'y/n');
  if(key == 'y' || key == 'Y'){
    alert("반복합니다.");
  }else if(key == 'n' || key == 'N'){
    alert("중단합니다.");
    break;
  }else{
    alert("잘못된 입력입니다.")
  }
}
console.log("프로그램 종료")
```

for 문

- **for문**

for(초기값; 조건식; 증감값){
 조건식이 true면 반복실행;
}

```
var i;
      ①          ②          ④
for(i = 1;  i <= 10; i++) {
                  ③
        console.log(i);
}
```

for 문

- **for문 – 1부터 5까지 더하기**

```
//1부터 10까지 출력
for(var i = 1; i <= 10; i++){
  console.log(i);
}

//1부터 5까지 더하기
let sum = 0;
for(var i=1; i<=5; i++){
  sum += i;
  console.log(`i=${i}, sum=${sum}`);
}
console.log(sum);
```

for 문

- **for문 – 1부터 입력값까지 더하는 프로그램**

합계 구하기

1부터 5까지의 합은 25입니다.

```
<h2>합계 구하기</h2>
<div id="result"></div>

<script src="js/sum.js"></script>
```

for 문

- **for문 – 1부터 입력값까지 더하는 프로그램**

```
const result = document.getElementById("result");

let num = prompt("몇까지의 합을 계산할까요?", "1");
let sum = 0;

num = parseInt(num);
for(var i = 1; i <= num; i++){
  sum += num;
}
result.textContent = `1부터 ${num}까지의 합은 ${sum}입니다.`;
```

배열(객체)이란?

배열 사용의 필요성

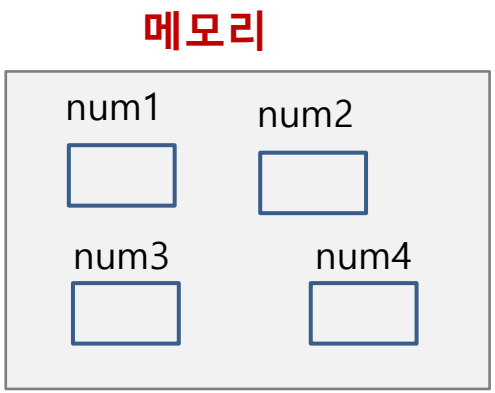
- 정수 10개를 이용한 학생의 성적 프로그램을 만들때

  10개의 변수를 선언

  var num1, var num2, var num3... var num10;

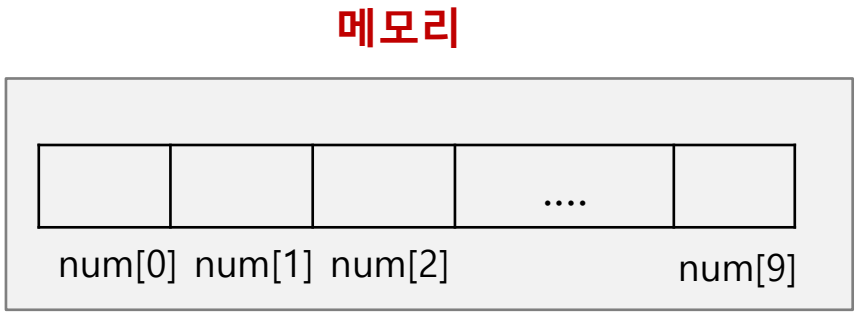
  정보가 흩어진 채 저장되고, 변수 이름이 많아

  비효율적이고 관리하기 어렵다.



배열 사용의 장점

- 인덱스를 이용하여 순차(순서)적으로 관리할 수 있다 -> 효율적이다.



배열(객체)이란?

배열(Array)이란?

- 여러 개의 연속적인 값을 저장하고자 할 때 사용하는 자료형이다.

배열의 생성

방법1 : let 배열 이름 = [값1, 값2, 값3...]

방법2 : let 배열 이름 = new Array(5)

문자형 배열

배열 생성 및 초기화

배열(Array)

첫 번째 과일: 사과
세 번째 과일: 포도

과일의 총 개수: 4

과일 1: 사과
과일 2: 배
과일 3: 포도
과일 4: 바나나

과일 1: 사과
과일 2: 배
과일 3: 포도
과일 4: 바나나
과일 5: 오렌지

과일 1: 사과
과일 2: 딸기
과일 3: 포도
과일 4: 바나나

문자형 배열

배열 생성 및 초기화

```
<h2>배열(Array)</h2>
<div id="result"></div>

<script>
  let result = document.getElementById("result");
  // 배열 생성
  let fruits = ["사과", "배", "포도", "바나나"];

  // 배열 요소 접근
  result.innerHTML += "첫 번째 과일: " + fruits[0] + "<br>"; // 사과
  result.innerHTML += "세 번째 과일: " + fruits[2] + "<br>"; // 포도
  result.innerHTML += "<br>";

  // 배열 길이
  result.innerHTML += "과일의 총 개수: " + fruits.length + "<br>";
  result.innerHTML += "<br>";
```

문자형 배열

배열 생성 및 초기화

```
// 배열 요소 출력
for(let i = 0; i < fruits.length; i++) {
  //result.innerHTML += "과일 " + (i + 1) + ": " + fruits[i] + "<br>";
  result.innerHTML += `과일 ${i + 1}: ${fruits[i]}<br>`;
}
result.innerHTML += "<br>";

// 배열 요소 추가
fruits.push("오렌지");

// 배열 요소 출력 (추가 후)
for(let i = 0; i < fruits.length; i++) {
  result.innerHTML += `과일 ${i + 1}: ${fruits[i]}<br>`;
}
result.innerHTML += "<br>";

// 배열 요소 제거
fruits.pop(); // 마지막 요소 제거
```

문자형 배열

배열 생성 및 초기화

```
// 배열 요소 변경
fruits[1] = "딸기"; // 두 번째 요소 변경

// 배열 요소 출력 (변경 후)
for(let i = 0; i < fruits.length; i++) {
  result.innerHTML += `과일 ${i + 1}: ${fruits[i]}<br>`;
}

</script>
```

숫자형 배열

Array 클래스를 사용한 배열

Array 객체를 사용한 배열

10
20
30
40

```
<h2>Array 클래스를 사용한 배열</h2>
<div id="demo"></div>

<script src="./js/array2.js"></script>
```

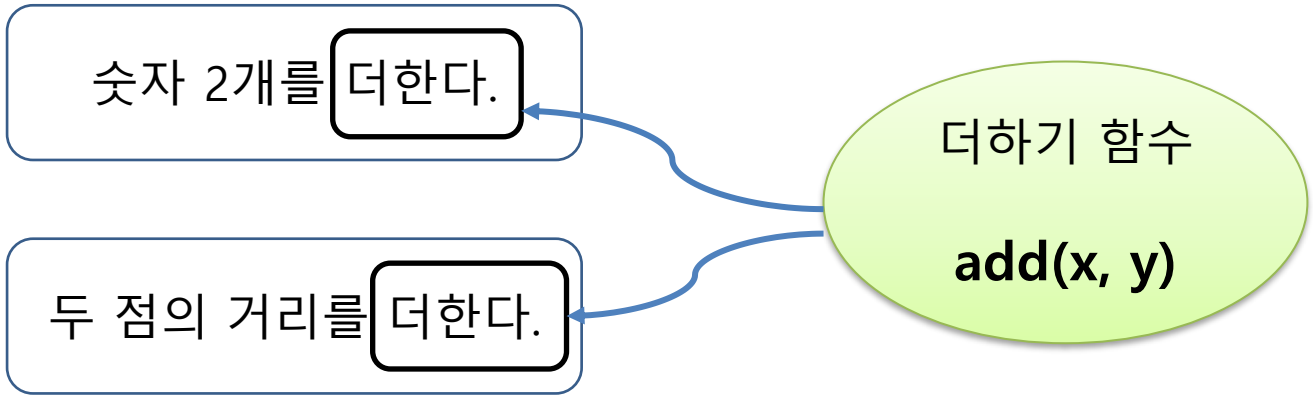
숫자형 배열

Array 객체를 사용한 배열

```
// 배열 생성(길이가 4인 배열)
let numbers = new Array(4);
// 배열에 값 할당(인덱스)
// numbers[0] = 10;
// numbers[1] = 20;
// numbers[2] = 30;
// numbers[3] = 40;

// for문을 이용하여 배열에 값 할당
for (let i = 0; i < numbers.length; i++) {
  numbers[i] = (i + 1) * 10;
}

// 배열 출력
for (let i = 0; i < numbers.length; i++) {
  demo.innerHTML += numbers[i] + "<br>";
}
```

함수의 정의

- **함수(Function)**
  - 특정한 하나의 기능을 수행하는 일련의 코드
  - 중복되는 기능은 함수로 구현하여 그 함수를 호출하여 사용한다.



함수 만들기

- **함수 (정의와 호출)**

명시적 함수(이름있음)

```
function 함수이름(){

        실행문;
}
```

함수 호출: **함수이름();**

함수의 유형

- 함수의 유형

반환값이 없는 경우

```
<h2>반환값이 없는 함수</h2>
<div id="result"></div>
<div id="result2"></div>

<h3>구구단 출력</h3>
<button onclick="gugudan(6)">확인</button>
<div class="display"></div>

<script src="js/func1.js"></script>
```

함수의 유형

- 함수의 유형

```
let result1 = document.getElementById("result");
let result2 = document.getElementById("result2");

//함수 정의
function sayHello(){
  result.innerTEXT = "안녕~";
}

function sayHello2(name){
  result2.innerTEXT = `안녕~ ${name}`;
}

//호출
sayHello();

sayHello2("영우");
sayHello2("Elsa")
```

함수의 유형

- 함수의 유형

```
//구구단
let display = document.querySelector(".display")
display.innerHTML = "<br>";

function gugudan(x){ //x는 매개변수
  for(let i = 1; i <= 9; i++){
    display.innerHTML += `${x} x ${i} = ${x*i}<br>`
  }
}
```

함수의 유형

- 함수의 유형

반환값이 있는 경우 – return 키워드 사용

반환값이 있는 함수

4의 제곱: 16

-10의 절대값: 10

응원 메시지

확인

Good Luck!!

```
<h2>반환값이 있는 함수</h2>
<p id="result"></p>
<p id="result2"></p>

<h3>응원 메시지</h3>
<button onclick="showMessage()">확인</button>
<div class="display"></div>

<script src="js/returnFunc.js"></script>
```

함수의 유형

- 함수의 유형

반환값이 있는 경우 – return 키워드 사용

```
//객체에 접근
let result = document.getElementById("result")
let result2 = document.getElementById("result2");
let display = document.querySelector(".display");

//제곱수 계산 함수
function square(x){
  return x * x;
}

//절대값 계산 함수
function myAbs(n){
  if(n < 0)
    return -n;
  else
    return n;
}
```

함수의 유형

- 함수의 유형

반환값이 있는 경우 – return 키워드 사용

```
//메시지 출력
function showMessage(){
  return display.innerHTML = "<em>Good Luck!!</em>";
}

//함수 호출
let num = square(4);
result.textContent = "4의 제곱: " + num;

let val = myAbs(-10);
result2.textContent = "-10의 절대값: " + val;
```

함수 만들기

- **익명 함수 만들기(정의와 호출)**

```
let 변수이름 = function(){

        실행문;
}
```

⇩

함수 호출: **변수이름();**

익명 함수

- 익명 함수

```html
<h2>익명 함수</h2>
<div id="result"></div>
<div id="result2"></div>

<script src="js/func2.js"></script>
```

```js
let greeting = function(){
  document.getElementById("result").innerText = "안녕하세요~";
}

let greeting2 = function(name){
  document.getElementById("result2").innerText = "안녕~ " + name;
}

greeting(); //호출

greeting2("선화");
```

화살표 함수

- **화살표 함수 만들기(정의와 호출)**

```
const 변수이름 = ()=>{
        실행문;
}
```

함수 호출: **변수이름();**

- 매개변수 없는 경우: () => { }
- 매개변수 1개인 경우: (x) => { }
- return이 있는 함수:

  (x, y) => { return x + y }

화살표 함수

- 화살표 함수

화살표 함수

안녕하세요~
안녕~ 상진

두 수의 합

계산

덧셈 결과: 30

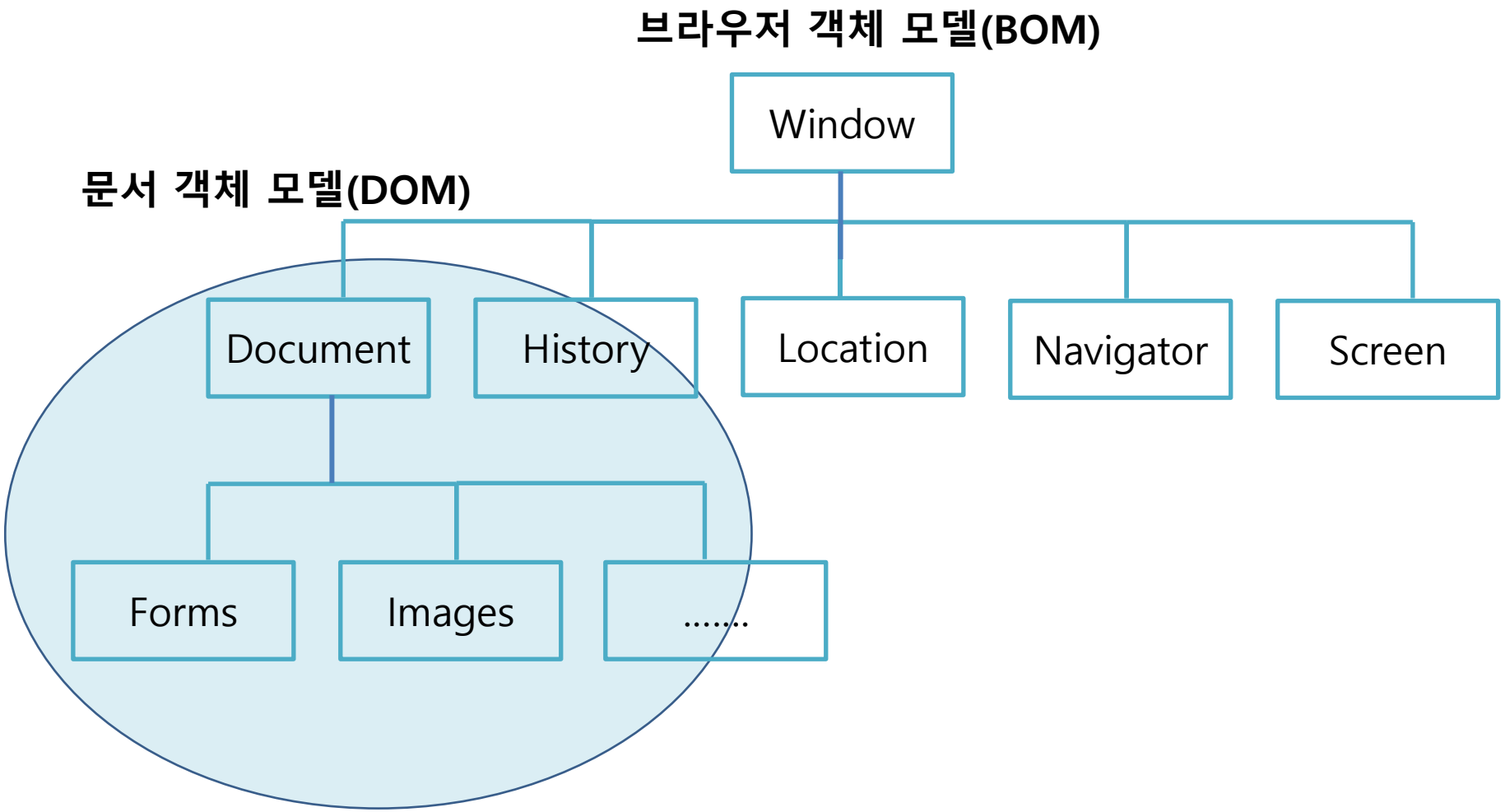
```
<h2>화살표 함수</h2>
<div id="result"></div>
<div id="result2"></div>

<h3>두 수의 합</h3>
<button onclick="showAdd(10, 20)">계산</button>
<p class="total"></p>

<script src="js/arrow.js"></script>
```

화살표 함수

- **화살표 함수**

```
let total = document.querySelector(".total");

const greeting = () => {
  document.getElementById("result").innerText = "안녕하세요~";
}

const greeting2 = (name) => {
  document.getElementById("result2").innerText = `안녕~ ${name}`;
}

//덧셈 함수
const add = (x, y) => {
  return x + y;
};

//add()를 호출
const showAdd = (a, b) => {
  total.textContent = `덧셈 결과: ${add(a, b)}`
}

//함수 호출
greeting();
greeting2("상진");
```

DOM(Document Object Model)



DOM(Document Object Model)

DOM이란?

웹 페이지(HTML 문서)를 **자바스크립트로 조작할 수 있도록 객체 모델 형태로 표현한 것**을 말합니다.

DOM은 HTML 문서를 객체 트리로 바꾼 것이고, 자바스크립트를 통해 DOM을 조작하면 **웹 페이지의 구조, 내용, 스타일, 동작을 동적으로 바꿀 수 있다.**

DOM 주요 객체

- **document** : 웹 페이지 전체를 대표하는 객체
- **element** : HTML 태그 하나하나를 객체로 표현
- **attribute** : 태그의 속성(id, class 등)
- **text** : 태그 안의 글자

문서 객체 모델(DOM)

DOM 요소에 접근하기

1. DOM 요소를 id 선택자로 접근하는 함수

```
document.getElementById("display")
```

속성	설명
innerHTML	태그 요소의 내용(텍스트)을 바꾸는 속성
style	태그 요소의 스타일을 바꾸는 속성
src	파일의 경로

문서 객체 모델(DOM)

DOM 요소에 접근하기

2. DOM 요소를 태그, id, class 선택자로 접근하는 함수

```
document.querySelector("img")
document.querySelector("#pic")
document.querySelector(".pic")
```

3. 객체.querySelectorAll(리스트) – 여러 개의 값을 반환

```
document.querySelectorAll(".city")
```

이벤트 다루기

이벤트 효과

이벤트(Event)란 웹 브라우저나 사용자가 실행하는 어떤 동작을 말한다.

마우스 이벤트

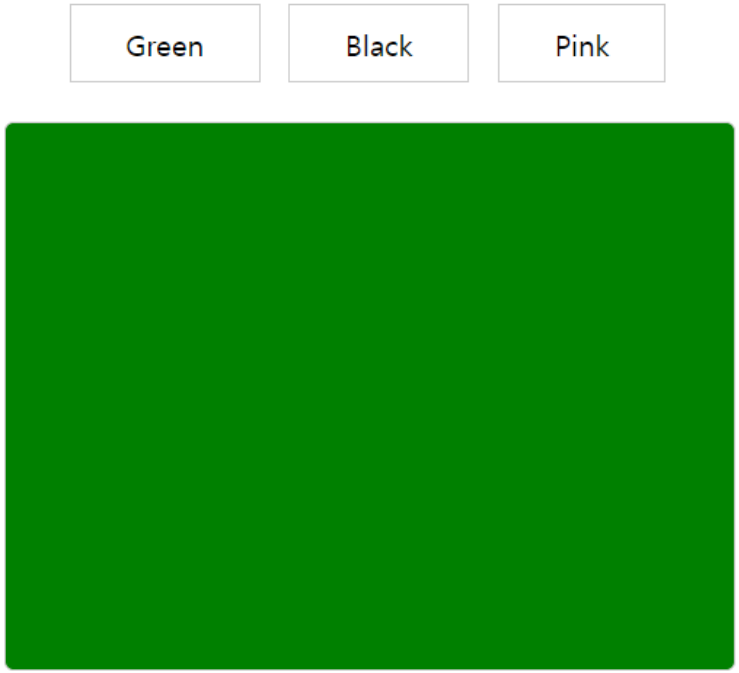
속성	설명
onclick	사용자가 HTML 요소를 클릭할 때 이벤트가 발생
onchange	목록이나 체크 상태 등이 변경되었을때 이벤트 발생
mouseover	마우스 포인터가 요소 위로 옮겨졌을때 이벤트 발생

키보드 이벤트

속성	설명
keypress	키를 눌렀을때 이벤트가 발생
keydown	키를 누르고 있는 동안 이벤트 발생
keyup	키에서 손을 뗄 때 이벤트 발생

태그 요소에 접근

- 버튼을 클릭하여 배경색 넣기



Green 클릭 -> 배경색 녹색

Black 클릭 -> 배경색 검정색

Pink 클릭 -> 배경색 분홍색

태그 요소에 접근

changebg.html

```
<!-- 버튼 메뉴 만들기 -->
<div class="menu">
  <ul>
    <li>
      <a href="#" onclick="changeBg('green')">Green</a>
    </li>
    <li>
      <a href="#" onclick="changeBg('black')">Black</a>
    </li>
    <li>
      <a href="#" onclick="changeBg('pink')">Pink</a>
    </li>
  </ul>
</div>
<!-- 배경색을 넣을 사각형 -->
<div id="rect" class="box"></div>

<script>
  function changeBg(color){
    let rect = document.querySelector("#rect");
    rect.style.backgroundColor = color;
  }
</script>
```

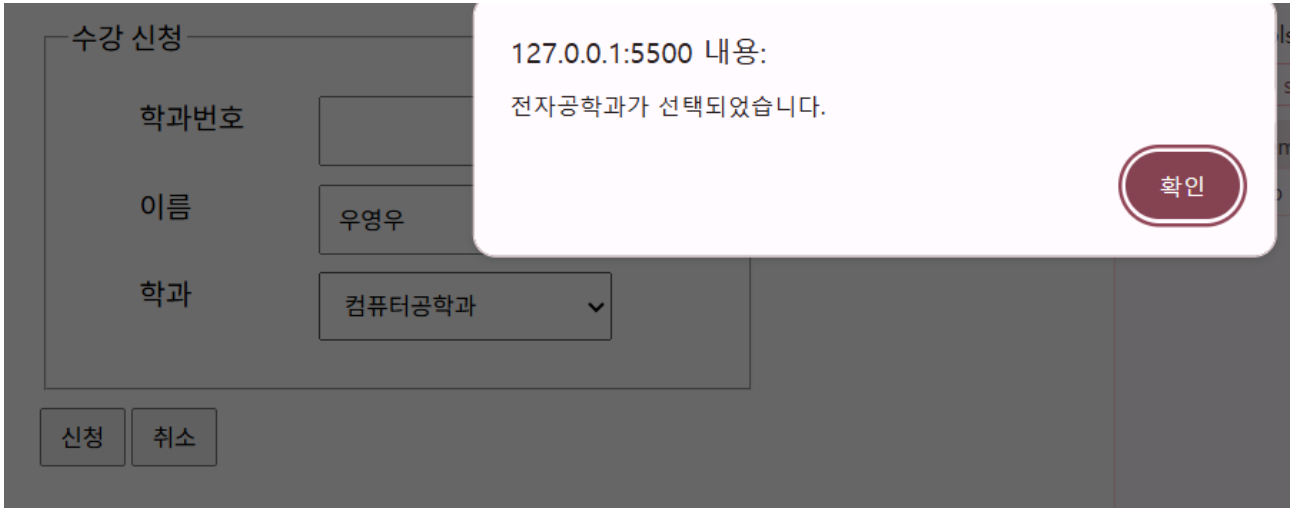
태그 요소에 접근

```
.menu{width: 400px; margin-left: 80px;}
.menu ul{list-style: none; padding: 0;}
.menu ul li{
  display: inline-block;
  margin: 10px;
  border: 1px solid #ccc;
  border-radius: 3px;
  padding: 10px 20px;
}
.menu ul li a{text-decoration: none; color: #000;}
.box{
  width: 400px;
  height: 300px;
  border: 1px solid #ccc;
  margin-left: 50px;
}
```

bgColor.css

선택한 옵션 항목 알림

- onchange 이벤트



선택한 옵션 항목 알림

- **onchange 이벤트**

subject.html

```
<li>
  <label for="sname">이름</label>
  <input type="text" id="sname" name="sname"
    value="우영우" readonly>
</li>
<li>
  <label for="major">학과</label>
  <select name="major" id="major" onchange="selectMajor()">
    <option value="컴퓨터공학과">컴퓨터공학과</option>
    <option value="전자공학과">전자공학과</option>
    <option value="기계공학과">기계공학과</option>
    <option value="화학공학과">화학공학과</option>
  </select>
</li>
```

선택한 옵션 항목 알림

- **onchange 이벤트**

major.js

```
// 학과 선택 시 알림을 띄우는 함수
function selectMajor() {
  const major = document.getElementById("major").value;
  alert(`${major}가 선택되었습니다.`);
}
```

객체의 정의

- **객체(Object)란?**

  - 프로그램에서 인식할 수 있는 모든 대상
  - 데이터를 저장하고 처리하는 기본 단위.

◎ **자바 스크립트 내장 객체**

문서 객체 모델(DOM) : 문서 뿐만 아니라 웹 문서 안에 포함된 이미지, 링크, 텍스트 필드 등을 모두 별도의 객체로 관리

브라우저 객체 모델(BOM) : 웹 브라우저 정보를 객체로 관리

◎ **사용자 정의 객체**

여러 가지 자료형을 포함하는 **'복합' 자료형**을 직접 만들어 사용하는 것을 말한다.

객체 지향 언어에서는 클래스(class)라고도 하며 객체의 설계도, 틀 역할을 한다.

내장 객체 – Date

Date 객체

- 날짜와 시간 정보를 다루는 객체
- **const date = new Date()**로 인스턴스를 만듬.

함수	설명
getFullYear()	연도를 4자리 숫자로 표시함
getMonth()	0부터 11사이의 숫자로 월을 표시함(0- 1월)
getDate()	1~31 사이의 숫자로 일을 표시함
getDay()	0~6 사이의 숫자로 요일을 표시함(0-일요일)
getHours()	0~23 사이의 숫자로 시를 표시함
getMinutes()	0~59 사이의 숫자로 분을 표시함
getSeconds()	0~59 사이의 숫자로 초를 표시함
getTime()	1970년 1월 1일 이후의 시간을 밀리 초로 표시함

내장 객체 – Date

Date 객체

```
// 현재 날짜와 시간
const now = new Date();
console.log("현재 날짜와 시간:", now);

// 날짜 정보 추출
console.log("연도:", now.getFullYear());
// 월은 0부터 시작하므로 +1
console.log("월:", now.getMonth() + 1);
console.log("일:", now.getDate());
// 0: 일요일, 1: 월요일, ..., 6: 토요일
console.log("요일:", now.getDay());

// 시간 정보 추출
console.log("시:", now.getHours());
console.log("분:", now.getMinutes());
console.log("초:", now.getSeconds());
console.log("현재까지의 시간:", now.getTime()/1000, "초");
```

```
현재 날짜와 시간: Wed Mar 18 2026 15:48:25 GMT+0900
연도: 2026
월: 3
일: 18
요일: 3
시: 15
분: 48
초: 25
현재까지의 시간: 1773816505.338 초
```

내장 객체 – window

- **window 객체**

  - **setInterval(in milliseconds)** : 초가 설정되고 계속 반복함.

  - **setTimeout(in milliseconds)** : 초가 설정되고 종료함

```
<h2>Window 객체 예제</h2>

<script>
  // 1초마다 실행되는 코드
  //window.setInterval() 또는 setInterval()로 사용
  setInterval(() => {
    console.log("1초마다 실행되는 코드");
  }, 1000);

  // 3초 후에 알림을 띄우는 코드
  setTimeout(() => {
    alert("3초가 지났습니다!");
  }, 3000);

</script>
```

디지털 시계

- 시계 표시하기

오후 4:06:07

```
<div class="content">
  <div id="display"></div>
</div>

<script src="./js/watch.js"></script>
```

```
<style>
  .content {
    text-align: center;
    margin-top: 50px;
  }
  #display {
    font-size: 48px;
    font-weight: bold;
  }
</style>
```

디지털 시계

- 시계 표시하기

```
// 시계 기능을 구현하는 코드
// display 요소 접근
let display = document.getElementById("display");

setInterval(() => {
  let now = new Date();
  let time = now.toLocaleTimeString();
  display.innerHTML = time;
}, 1000);
```

객체의 정의

- **사용자 정의 객체**
  - 여러가지 자료형을 포함하는 '복합' 자료형을 말한다.
  - 객체는 속성과 메서드로 구성되어 있다.

속성(property)
나이 : 26
이름 : 이정후
결혼유무 : false

메서드(function)
야구를 한다.

사람

```
let 객체이름 = {

속성 이름 : 값,
속성 이름 : 값,
함수 이름 : function(){...}

}
```

콜론(:)사용 — 콤머(,)사용

키(key) : 값(value) 형태

객체의 정의 및 사용

- 사용자 정의 객체(object)

```
<h2>Person 객체</h2>
<div id="demo"></div>

<script>
  let person = {
    name: "이정후",
    age: 27,
    play: function() {
      return `${this.name}가 야구를 합니다.`;
    }
  };

  //document.getElementById("demo").innerHTML = person.play();
  document.getElementById("demo").innerHTML = `
    이름: ${person.name} <br>
    나이: ${person.age} <br>
    활동: ${person.play()}
  `;
</script>
```

Person 객체

이름: 이정후
나이: 27
활동: 이정후가 야구를 합니다.

객체의 정의 및 사용

- 여러 명의 객체(object) – 객체 배열

Person 객체 배열

이정후

나이: 27,
스포츠: 야구

최민정

나이: 27,
스포츠: 스피드 스케이팅

이강인

나이: 25,
스포츠: 축구

```
<h2>Person 객체 배열</h2>
<div id="demo"></div>

<script src="../js/persons.js"></script>
```

객체의 정의 및 사용

- 여러 명의 객체(object) – 객체 배열

```
const persons = [
  {
    name: '이정후',
    age: 27,
    sport: '야구',
    play: function() {
      console.log(`${this.name}이(가) ${this.sport}을(를) 합니다.`);
    }
  },
  {
    name: '최민정',
    age: 27,
    sport: '스피드 스케이팅',
    play: function() {
      console.log(`${this.name}이(가) ${this.sport}을(를) 합니다.`);
    }
  },
  {
    name: '이강인',
    age: 25,
    sport: '축구',
    play: function() {
      console.log(`${this.name}이(가) ${this.sport}을(를) 합니다.`);
    }
  }
```

객체의 정의 및 사용

- 여러 명의 객체(object) – 객체 배열

```
// HTML 요소에 정보 출력
demo.innerHTML = `
  <h3>${person.name}</h3>
  나이: ${person.age},<br>
  스포츠: ${person.sport}
`;

// 객체 배열 순회
for (let i = 0; i < persons.length; i++) {
    const person = persons[i];
    demo.innerHTML += `
    <h3>${person.name}</h3>
    나이: ${person.age},<br>
    스포츠: ${person.sport}
  `;
}
```

객체의 정의 및 사용

- 여러 명의 객체(object) – 객체 배열

```
// forEach 메서드 사용
persons.forEach(person => {
    demo.innerHTML += `
    <h3>${person.name}</h3>
    나이: ${person.age},<br>
    스포츠: ${person.sport}
  `;
});
```

Node 설치 및 실행

- **Node.js란?**

  Node.js는 구글의 V8 JavaScript 엔진을 기반으로 서버 측에서 JavaScript를 실행할 수 있도록 만든 **오픈소스 런타임 환경**입니다.

  (**웹 브라우저 밖에서도 javascript를 실행**)

- **서버 작업**

  - 웹 서버 만들기
  - API 서버 만들기
  - 파일 처리
  - 데이터베이스 연동

Node 설치 및 실행

- **Node.js 특징**
  - 비동기(Asynchronous) 처리
  - 싱글 스레드 이벤트 루프
  - Npm(Node Package Manager) 패키지 관리자 사용
    (패키지 – 배포할수 있도록 여러 모듈과 모듈 관련 파일을 묶어 놓은 것)

§ 서버 예시 코드

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.write("Hello Node.js");
  res.end();
});

server.listen(3000);
```

백엔드 vs 프런트엔드 프레임워크

- **Express 프레임워크**

  Node.js 기반에서 동작하는 **경량 웹 프레임워크(Web Framework)**입니다.
  주로 **웹 서버와 REST API를 빠르게 구축**하기 위해 사용되며, Node.js 생태계에서 가장 많이 사용되는 서버 프레임워크 중 하나입니다.

- **React 프레임워크**

  React(리액트)는 사용자 인터페이스(UI)를 만들기 위한 **JavaScript 라이브러리** 입니다.
  2013년에 **Meta Platforms (구 Facebook)**에서 공개했으며, 현재 웹 프론트엔드 개발에서 가장 널리 사용되는 기술 중 하나입니다.
  주로 **웹 애플리케이션의 화면(UI)**을 효율적으로 만들기 위해 사용됩니다.

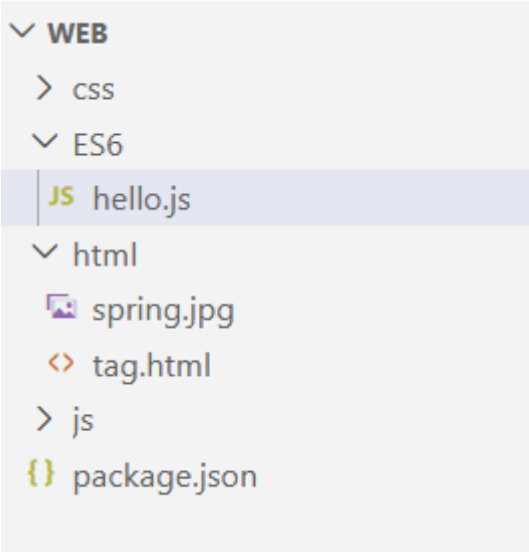
Node 설치 및 실행

- **Node js 다운로드 및 설치**

- **설치 확인 : node -v**

javascript 파일 실행

- **hello.js 만들고 실행하기**




```
// hello 함수 정의
function hello(name) {
    console.log(`Hello, ${name}!`);
}

// hello 함수 호출
hello('World');
```

☞ 터미널에서 실행

₩web₩es6> node hello

javascript ES6

- **ES6(ECMAScript 2015)**

  ES6(ECMAScript 2015)는 자바스크립트의 큰 업데이트로, 코드의 **가독성, 모듈화, 생산성**을 크게 개선한 문법과 기능들이 추가되었습니다.
  현대 프론트엔드 개발(특히 **React**, **Node.js**)에서 기본적으로 사용되는 표준 문법입니다.

- **ES6(ECMAScript 2015) 주요 특징**

  1. let / const (변수 선언)
  2. Template Literal (템플릿 문자열)
  3. Arrow Function (화살표 함수)
  4. Destructuring (구조 분해 할당)
  5. Spread / Rest 연산자 (...)
  6. Module (모듈 시스템) – export/import(이전 – commons, require()사용)

javascript ES

- **템플릿 문자열 – literal.js**

```
//템플릿 문자열
let season = '봄';

//연결 연산자 - '+'
console.log('현재 계절은 ' + season + '입니다.');

//백틱(`)으로 감싸고, ${}로 변수나 표현식을 감싸면 된다.
console.log(`현재 계절은 ${season}입니다.`);
```

javascript ES

- **화살표 함수 – function01.js**

```
//함수 선언
let add = function (a, b) {
  return a + b;
}

console.log(add(10, 20));

//화살표 함수
let add2 = (a, b) => {
  return a + b;
}

console.log(add2(10, 20));
```

javascript ES

- **화살표 함수 – function01.js**

```
//매개변수가 하나인 경우, 괄호 생략 가능
let square = x => {
  return x * x;
}

/*매개변수가 하나이고, 함수 본문이 한 줄인 경우,
  중괄호와 return 생략 가능*/
let square2 = x => x * x;

console.log(square(5));
console.log(square2(5));

//매개 변수 없는 경우
let message = () => console.log('Good, Luck!');
message();
```

javascript ES

- **구조 분해 할당 – destructure.js**

```
//배열 구조 분해 할당
const arr = [1, 2];
//console.log(arr[0]); //1
//console.log(arr[1]); //2

const [a, b] = arr; //
console.log(`a: ${a}`);
console.log(`b: ${b}`);


//객체 구조 분해 할당
const product = {
  prod_name: '무선마우스',
  price: 27000
};

const { prod_name, price } = product;
console.log(`제품명: ${prod_name}`);
console.log(`가격: ${price}`);
```

javascript ES

- **Spread / Rest 연산자 (...)**

```
//배열에서 사용
let arr1 = [1, 2, 3];
let arr2 = [4, 5];

//배열을 펼쳐서 새로운 배열을 만든다.
let newArr = [...arr1, ...arr2];
console.log(newArr);

//객체에서 사용
let obj1 = { product: '무선마우스', price: 27000 };
let obj2 = { spec: 'M200 무선마우스 그레이' };

//객체를 펼쳐서 새로운 객체를 만든다.
let combinedObj = { ...obj1, ...obj2 };
console.log(combinedObj);
```

```
[ 1, 2, 3, 4, 5 ]
{ product: '무선마우스', price: 27000, spec: '네이버 M200 무선마우스 그레이' }
```

javascript ES

- **Array Methods - map() 함수**

  배열의 각 요소에 대해 주어진 함수를 호출한 결과를 모아 **새로운 배열**을 반환하는 함수

```
const arr = [1,2,3];

const newArr = arr.map(x => x * 2);
console.log(newArr); // [2, 4, 6]

const users = [
  { name: 'Jerry', age: 25 },
  { name: 'Linda', age: 30 },
  { name: 'Tom', age: 35 }
];

const names = users.map(user => user.name);
console.log(names); // ['Jerry', 'Linda', 'Tom']

const ages = users.map(user => user.age);
console.log(ages); // [25, 30, 35]
```

javascript ES

- **ES 모듈 시스템(export / import)**

  package.json에 'type' 속성 추가해야 함

  package.json은 **npm init –y** 명령어로 설치할 수 있음

```
{
  "name": "web",
  "version": "1.0.0",
  "main": "index.js",
  ▷디버그
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "type": "module",
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "ansi-colors": "^4.1.3"
  }
}
```

javascript ES

- **lib/myAbs.js**

```
function myAbs(x) {
  if (x < 0) {
    return -x;
  } else {
    return x;
  }
}

export { myAbs }; // export { myAbs, square }; 배열로 내보내기
// export default myAbs;
```

- **main.js**

```
import { myAbs } from "./lib/myAbs.js";

console.log(myAbs(-5));    // 5
console.log(myAbs(10));    // 10
```

javascript ES

- **비동기 처리**

  자바스크립트는 싱글 스레드 방식으로 처리해야 할 기능을 순차적으로 처리함. 비동기 처리는 프로그램에서 처리해야 할 기능들을 순차적으로 처리하는 것이 아니라, 원하는 순서에 맞게 처리하는 것

방식	버전	기능
콜백 함수	기존부터 사용	함수 안에 또 다른 함수를 매개변수로 넘겨서 실행 순서를 제어함
프로미스(Promise)	에크마스크립트 2015	프로미스 객체와 콜백 함수를 사용해서 실행 순서를 제어함
async/await	에크마스크립트 2017	async와 await를 사용해서 실행순서 제어

javascript ES

- **동기 처리 – async01.js**

```javascript
function displayA() {
  console.log('A');
}

function displayB() {
  // console.log('B');
  // 2초 후에 'B'를 출력하는 비동기 처리
  setTimeout(() => {
    console.log('B');
  }, 2000);
}

function displayC() {
  console.log('C');
}

displayA();
displayB();
displayC();
```

```javascript
/*싱글 스레드이므로 동기적으로 실행됨
  displayA() -> displayB() -> displayC() 순서로 실행됨
  displayB()는 2초 후에 'B'를 출력하는 비동기 처리이므로,
  displayC()가 먼저 실행되어 'C'가 출력됨.
  'A' -> 'C' -> 'B' 순서로 출력됨*/
```

```
A
C
B
```

javascript ES

- **비동기 처리 – async02.js**

```javascript
function displayA() {
  console.log('A');
}

function displayB(callBack) {
  setTimeout(() => {
    console.log('B');
    callBack();
  }, 2000);
}

function displayC() {
  console.log('C');
}

displayA();
displayB(displayC);
```

```javascript
/*
  비동기 처리 - 프로그램에서 처리해야 할 기능들을 순차적으로
  처리하는 것이 아니라, 동시에 처리하는 방식
  원하는 순서에 맞게 처리하기 위해 콜백함수를 사용
  'A' -> 'B' -> 'C' 순서로 출력됨
*/
```

```
A
B
C
```

javascript ES

- **비동기 처리 – async03.js**

```javascript
async function displayA() {
  console.log('A');
}

async function displayB() {
  // Promise 객체를 반환하여 비동기 작업 처리
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log('B');
      resolve();
    }, 2000);
  });
}

async function displayC() {
  console.log('C');
}
```

```javascript
/*
  비동기 처리 - async/await 방식
  async/await를 사용하여 더 깔끔하게 비동기 작업 처리
  Promise를 명시적으로 사용하지 않음
  'A' -> 'B' -> 'C' 순서로 출력됨
*/
```

javascript ES

- **비동기 처리 – async03.js**

```
// async/await를 사용하여 순차적으로 실행
async function runSequence() {
  await displayA();
  await displayB();
  await displayC();
}

runSequence(); // runSequence() 함수를 호출하여 실행
```

```
A
B
C
```